

CONFIDENTIAL — FOR AUTHORIZED TESTERS ONLY

# Product Testing & Vulnerability Assessment Guide

A complete guide for security engineers and developers to test their applications for post-quantum cryptographic vulnerabilities using the PQ-Sentry platform.



Version 1.0.0



PQ-Sentry Platform



17 May 2026



## TABLE OF CONTENTS

- [01 What is PQ-Sentry and Why You Need It](#)
- [02 The Quantum Threat — Understanding the Risk](#)
- [03 How PQ-Sentry Testing Works \(Workflow Overview\)](#)
- [04 Setting Up the Testing Environment](#)
- [05 Test Scenario A — Dashboard Vulnerability Discovery](#)
- [06 Test Scenario B — SBOM ↔ CBOM Correlation Audit](#)
- [07 Test Scenario C — AI-Powered Remediation Playbook](#)
- [08 Test Scenario D — Live Quantum Migration \(Core Test\)](#)
- [09 Test Scenario E — Compliance Audit Report Generation](#)
- [10 Test Scenario F — Ingesting Your Own Application Data](#)
- [11 API Reference for Custom Testing](#)
- [12 Vulnerability Classification Matrix](#)
- [13 Test Acceptance Criteria Checklist](#)
- [14 Resetting the Environment](#)
- [15 FAQ & Troubleshooting](#)

## [01 What is PQ-Sentry and Why You Need It](#)

---

**PQ-Sentry** is a Post-Quantum Cryptography Readiness Platform built for Kubernetes environments. It scans, inventories, and audits the cryptographic algorithms used by your microservices, then provides actionable migration paths to quantum-safe standards (NIST FIPS 203/204/205).

 **KEY VALUE PROPOSITION**

PQ-Sentry answers one critical question: **"If a quantum computer existed today, which of my services would be immediately compromised?"** — and then gives you the tools to fix it with zero application code changes.

**Core Capabilities**

| CAPABILITY                | DESCRIPTION                                                                     | TESTING FOCUS                                               |
|---------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>CBOM Inventory</b>     | Cryptographic Bill of Materials — real-time catalog of all cipher suites in use | Verify accurate algorithm detection and risk classification |
| <b>SBOM ↔ CBOM Bridge</b> | Correlates software dependency CVEs with cryptographic cipher vulnerabilities   | Verify unified risk scoring and CVE-to-cipher mapping       |
| <b>AI Risk Advisor</b>    | GenAI-powered remediation playbooks with Kubernetes-native manifests            | Verify playbook accuracy and actionability                  |
| <b>Quantum Migration</b>  | One-click cipher upgrade from classical → post-quantum (ML-KEM-768)             | Verify live database migration and dashboard refresh        |
| <b>Compliance Audits</b>  | Auto-generated reports for SOC2, FedRAMP, NIS2, CMMC 2.0                        | Verify regulatory checklist accuracy                        |

02

# The Quantum Threat — Understanding the Risk

Cryptographically Relevant Quantum Computers (CRQCs) threaten all public-key cryptography based on integer factorization (RSA) and discrete logarithms (ECDSA). Using **Shor's Algorithm**, a CRQC can break these in polynomial time.

## Vulnerability Classification

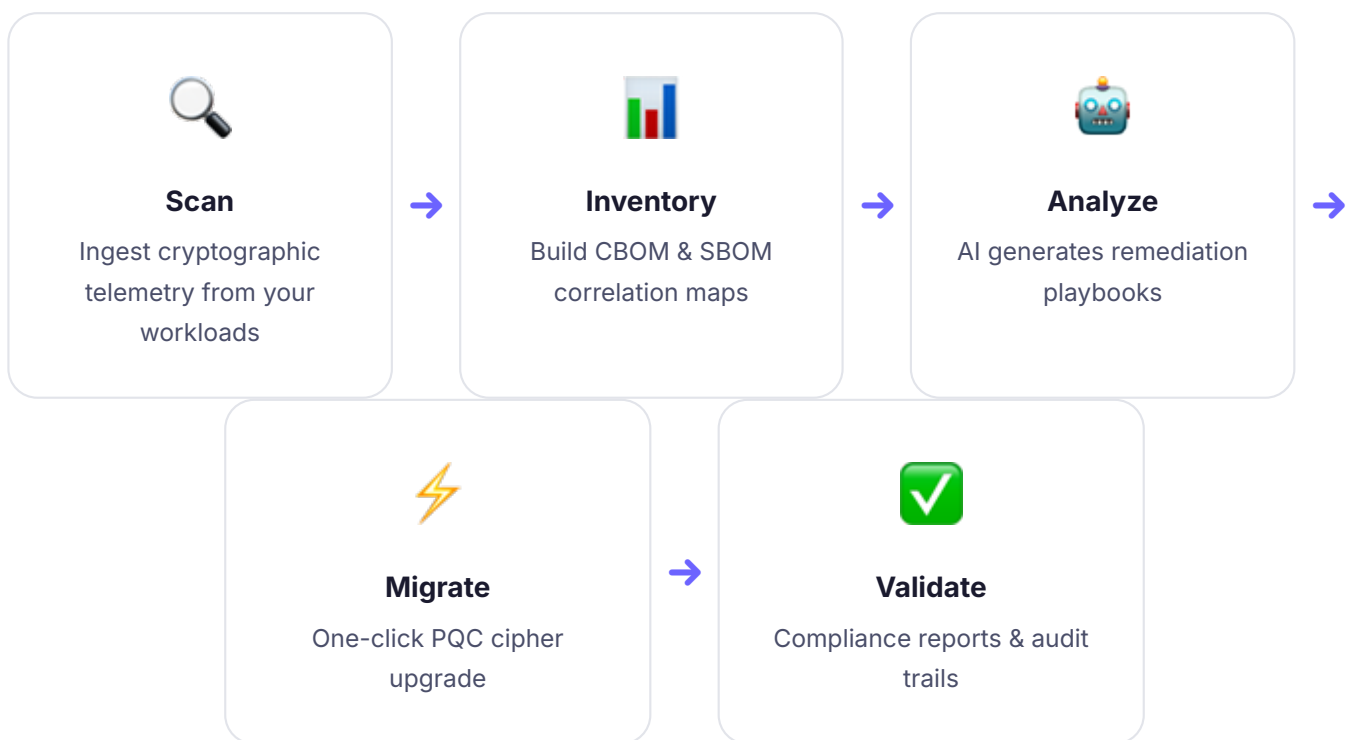
| Algorithm        | Type                   | Key Size | Quantum Risk | NIST Replacement                                    |
|------------------|------------------------|----------|--------------|-----------------------------------------------------|
| RSA-2048         | Asymmetric (Factoring) | 2048     | CRITICAL     | ML-KEM-768 (FIPS 203)                               |
| RSA-4096         | Asymmetric (Factoring) | 4096     | CRITICAL     | ML-KEM-1024 (FIPS 203)                              |
| ECDSA P-256      | Elliptic Curve (DLOG)  | 256      | HIGH         | ML-DSA-65 (FIPS 204)                                |
| ECDSA P-384      | Elliptic Curve (DLOG)  | 384      | HIGH         | ML-DSA-87 (FIPS 204)                                |
| AES-256-GCM      | Symmetric              | 256      | SAFE         | No change (Grover: $\sqrt{\phantom{x}}$ complexity) |
| ML-KEM-768       | Lattice-based (PQC)    | 768      | PQC SAFE     | Already quantum-resistant                           |
| ML-DSA-65        | Lattice-based (PQC)    | n/a      | PQC SAFE     | Already quantum-resistant                           |
| None / Plaintext | Unencrypted            | 0        | CRITICAL     | Immediate TLS required                              |

### ⚠️ HARVEST NOW, DECRYPT LATER (HNDL)

Adversaries are **already recording encrypted traffic today** to decrypt it once quantum computers are available. Services handling PII, financial transactions, health records, or classified data are at immediate risk — even before CRQCs exist.

## 03 How PQ-Sentry Testing Works

The testing workflow follows a 5-stage pipeline. Each stage produces verifiable outputs:



## 04 Setting Up the Testing Environment

### ✅ PREREQUISITES

You only need **Docker Desktop** (v24.x+). No Java, Maven, PostgreSQL, or build tools required.

## STEP 1

## Extract the Distribution Package

```
# Unzip the package you received
unzip pq-sentry-dist.zip
cd pq-sentry-dist
```

## STEP 2

## Launch PQ-Sentry

```
chmod +x start.sh stop.sh
./start.sh
```

Wait ~30-60 seconds until you see: 🚩 PQ-SENTRY IS READY FOR TESTING

## STEP 3

## Open the Dashboard

Open `dashboard/index.html` in Chrome, Firefox, or Edge. The dashboard connects to the local API on port 8080.

```
# macOS
open dashboard/index.html

# Linux
xdg-open dashboard/index.html
```

## STEP 4

## Verify API Health

```
curl http://localhost:8080/api/actuator/health
# Expected: {"status":"UP"}
```

05

# Test Scenario A — Dashboard Vulnerability Discovery

**Objective:** Verify that PQ-Sentry correctly identifies and classifies quantum-vulnerable microservices.

ACTION

Open the Dashboard Overview

Navigate to the Overview page (loads by default). Inspect the CBOM table listing all microservices.

Expected Results — Pre-Seeded Demo Data

| SERVICE      | NAMESPACE | ALGORITHM  | KEY SIZE | RISK LEVEL | QUANTUM SAFE?                                                                             |
|--------------|-----------|------------|----------|------------|-------------------------------------------------------------------------------------------|
| old-rsa      | payments  | RSA-2048   | 2048     | CRITICAL   |  No    |
| auth-service | auth      | ECDSA-P256 | 256      | HIGH       |  No    |
| new-pqc      | payments  | ML-KEM-768 | 768      | SAFE       |  Yes |

Validation Checklist

- ☐ Risk Score ring chart displays a score  $\geq 70$  (CRITICAL range)
- ☐ Total Services count = 3
- ☐ Critical count = 1 (old-rsa)
- ☐ High count = 1 (auth-service)
- ☐ Safe count = 1 (new-pqc)
- ☐ Each row has color-coded risk badges matching the table above
- ☐ Clicking the ✨ Consult button redirects to the AI Advisor



06

Test Scenario B — SBOM ↔ CBOM Correlation Audit

**Objective:** Verify that PQ-Sentry correctly correlates software package vulnerabilities (SBOM) with cryptographic posture (CBOM).

ACTION

Navigate to SBOM ↔ CBOM Bridge

Click "SBOM ↔ CBOM Bridge" in the left sidebar. The split-view panel loads correlated data.

Expected CVE Correlations

| SERVICE      | PACKAGE     | VERSION   | CVE           | SEVERITY | CIPHER RISK | UNITS SCORE |
|--------------|-------------|-----------|---------------|----------|-------------|-------------|
| old-rsa      | openssl     | 1.1.1     | CVE-2023-0286 | CRITICAL | CRITICAL    | 100         |
| old-rsa      | openssl     | 1.1.1     | CVE-2023-3817 | HIGH     | CRITICAL    | 100         |
| auth-service | boringssl   | fips-2021 | CVE-2022-0778 | HIGH     | HIGH        | 85          |
| new-pqc      | oqs-openssl | 3.1.0-pqc | None          | CLEAN    | SAFE        | 10          |

## Validation Checklist

- ☐ Split panel shows CBOM on left, SBOM on right
- ☐ CVE badges display with correct severity colors
- ☐ Unified risk score combines both SBOM + CBOM factors
- ☐ `new-pqc` shows clean status with no CVEs
- ☐ Correlation insights text is displayed for each workload

07

## Test Scenario C — AI-Powered Remediation Playbook

**Objective:** Verify the AI Advisor generates accurate, actionable remediation plans for vulnerable services.

### ACTION

#### Consult the AI Advisor

Click ✨ **Consult** next to `old-rsa` in the CBOM table (or navigate to AI Advisor in the sidebar). Select the service, then click **Consult AI Advisor**.

## Expected Playbook Contents

- ☐ **Vulnerability Summary** — Explains Shor's Algorithm risk for RSA-2048
- ☐ **NIST Compliance Target** — References FIPS 203 (ML-KEM-768) and FIPS 204 (ML-DSA-65)

- ☐ **YAML Manifest** — `QuantumReadinessPolicy` custom resource with:
  - Correct namespace matching the service
  - `algorithm: ML-KEM-768`
  - `signatureAlgo: ML-DSA-65`
  - Canary rollout strategy
- ☐ **kubectl Commands** — Executable deployment commands provided
- ☐ **Rendering** — Markdown renders with proper code blocks, headings, and formatting

## 08 Test Scenario D — Live Quantum Migration ★

### ★ CORE FEATURE TEST

This is the **primary test**. The migration engine actually updates all classical ciphers in the database to post-quantum standards and triggers a complete E2E dashboard refresh.

#### STEP 1

### Open the Policies Page

Click **Policies** in the left sidebar. You'll see a YAML editor and an operator terminal panel.

## STEP 2

## Configure the Migration

**Uncheck** the "Dry-Run Mode" checkbox. This enables the live migration mode.

### 💡 DRY-RUN VS. LIVE

**Dry-Run ON** = Simulates the migration without making changes. **Dry-Run OFF**  
= Actually migrates ciphers in the database.

## STEP 3

## Execute the Migration

Click ► **Apply Policy**. Watch the operator terminal.

## Expected Terminal Output

```
[INFO] Parsing QuantumReadinessPolicy: 'production-pqc'
[INFO] Rollout strategy configured: Canary
[INFO] Target namespaces: default, payments, auth, billing
[INFO] Scanning cbom_entries for quantum-vulnerable workloads...
[INFO] Upgrading old-rsa from RSA-2048 → ML-KEM-768 (PQC Safe)
[INFO] Injecting PQC Envoy sidecar into old-rsa pods...
[INFO] Upgrading auth-service from ECDSA-P256 → ML-KEM-768 (PQC Safe)
[INFO] Injecting PQC Envoy sidecar into auth-service pods...
[INFO] Recalculating unified cluster risk score...
[SUCCESS] 2 services upgraded to post-quantum cipher suites.
[SUCCESS] Cluster risk score updated: CRITICAL → SAFE (score: 0)
```

## Expected Dashboard Changes (Auto-Refresh)

| Metric                 | Before Migration | After Migration |
|------------------------|------------------|-----------------|
| Risk Score             | ≥ 70 (CRITICAL)  | 0 (SAFE)        |
| Critical Services      | 1                | 0               |
| High Services          | 1                | 0               |
| Safe Services          | 1                | 3               |
| old-rsa Algorithm      | RSA-2048         | ML-KEM-768      |
| auth-service Algorithm | ECDSA-P256       | ML-KEM-768      |
| Compliance Status      | WARN / PARTIAL   | COMPLIANT       |

Validation Checklist

- ☐ Terminal shows step-by-step log animation (typewriter effect)
- ☐ All 3 services now display ML-KEM-768 with SAFE badges
- ☐ Risk score ring chart updates to 0
- ☐ Overview metrics auto-refresh without manual reload
- ☐ SBOM ↔ CBOM Bridge reflects updated cipher suites
- ☐ Compliance audit indicators turn green
- ☐ Running migration again reports "No quantum-vulnerable workloads detected"

09

# Test Scenario E — Compliance Audit Report Generation

**Objective:** Verify that the platform generates accurate compliance reports for multiple regulatory frameworks.

ACTION

Generate and Export the Report

On the Overview page, click **Export PDF**. A new browser tab opens with the compliance report. Use `Ctrl+P` / `Cmd+P` to save as PDF.

Frameworks Covered

| FRAMEWORK | CONTROLS CHECKED           | PRE-MIGRATION STATUS | POST-MIGRATION STATUS |
|-----------|----------------------------|----------------------|-----------------------|
| SOC2 TSC  | CC6.1, CC6.7               | WARN                 | PASS                  |
| FedRAMP   | PQC Transition, FIPS 140-3 | FAIL / PARTIAL       | PASS                  |
| EU NIS2   | Art 21.1, Art 21.2         | WARN                 | PASS                  |
| CMMC 2.0  | SC.L2-3.13.11              | WARN                 | PASS                  |

Validation Checklist

- ☐ Report opens in new tab with styled, print-ready layout
- ☐ All four compliance frameworks are displayed
- ☐ Detailed CBOM inventory table is included
- ☐ Print/PDF output renders cleanly (white background, no cutoffs)

10

# Test Scenario F — Ingesting Your Own Application Data

---



## ADVANCED TESTING

You can ingest your own application's cryptographic data to see how PQ-Sentry would classify and handle your real services.

Use the `POST /api/cbom/{clusterId}/ingest` endpoint to push custom CBOM telemetry. This simulates what the PQ-Sentry CLI scanner would report from a live Kubernetes cluster.

## How to Audit Your Own Application

## STEP 1

## Identify Your Cryptographic Posture

Before ingesting, gather this information about each of your services:

| FIELD                    | DESCRIPTION                       | EXAMPLE VALUES                                                         |
|--------------------------|-----------------------------------|------------------------------------------------------------------------|
| <code>serviceName</code> | Name of your microservice         | <code>payment-gateway</code> , <code>user-api</code>                   |
| <code>namespace</code>   | Kubernetes namespace              | <code>production</code> , <code>staging</code>                         |
| <code>algorithm</code>   | TLS/cipher algorithm in use       | <code>RSA-2048</code> , <code>ECDSA-P256</code> , <code>AES-256</code> |
| <code>keySize</code>     | Key size in bits                  | <code>2048</code> , <code>256</code> , <code>384</code>                |
| <code>tlsEnabled</code>  | Whether TLS is active             | <code>true</code> / <code>false</code>                                 |
| <code>certExpiry</code>  | Certificate expiration (ISO 8601) | <code>2026-12-31T00:00:00Z</code>                                      |

## STEP 2

## Reset the Environment (Optional)

```
docker compose down -v
./start.sh
```



## STEP 3

## Ingest Your Service Data

```
curl -X POST http://localhost:8080/api/cbom/00000000-0000-0000-0000-0000-0000-0000-0000-0000 \
-H "Content-Type: application/json" \
-d '[
  {
    "serviceName": "payment-gateway",
    "namespace": "production",
    "algorithm": "RSA-2048",
    "keySize": 2048,
    "tlsEnabled": true,
    "certExpiry": "2026-12-31T00:00:00Z"
  },
  {
    "serviceName": "user-api",
    "namespace": "production",
    "algorithm": "ECDSA-P256",
    "keySize": 256,
    "tlsEnabled": true,
    "certExpiry": "2027-03-15T00:00:00Z"
  },
  {
    "serviceName": "internal-cache",
    "namespace": "infrastructure",
    "algorithm": "None",
    "keySize": 0,
    "tlsEnabled": false,
    "certExpiry": null
  }
]
```

## STEP 4

## View Results in Dashboard

Refresh [dashboard/index.html](#) . Your custom services now appear alongside the demo data. You can:

- See your services' risk classifications
- Run the AI Advisor against your specific services
- Apply the quantum migration to upgrade them to ML-KEM-768
- Generate compliance reports including your services

### TIP — TEST DIFFERENT SCENARIOS

Try ingesting services with various algorithms ( [RSA-4096](#) , [DES-56](#) , [3DES](#) , [AES-128](#) ) to see how PQ-Sentry classifies and prioritizes different cryptographic risk levels.

## 11 API Reference for Custom Testing

Base URL: <http://localhost:8080/api> | Default Cluster ID: [00000000-0000-0000-0000-000000000001](#)

| METHOD | ENDPOINT                        | DESCRIPTION                      | AUTH     |
|--------|---------------------------------|----------------------------------|----------|
| GET    | /actuator/health                | API health check                 | None     |
| GET    | /cbom/{clusterId}               | List all CBOM entries            | None     |
| GET    | /cbom/{clusterId}/summary       | Aggregated risk summary          | None     |
| POST   | /cbom/{clusterId}/ingest        | Ingest custom CBOM telemetry     | API Key* |
| GET    | /sbom/{clusterId}/bridge        | SBOM ↔ CBOM correlation          | None     |
| POST   | /ai/advise                      | Generate AI remediation playbook | None     |
| POST   | /policies/{clusterId}/apply     | Apply quantum migration          | None     |
| GET    | /reports/compliance/{clusterId} | Generate compliance report       | None     |

\* API Key auth is enforced in production. In the sandbox testing environment, ingestion is open for ease of testing.

12

Vulnerability Classification Matrix

Use this matrix to understand how PQ-Sentry classifies the algorithms found in your applications:

| RISK LEVEL | SCORE RANGE | ALGORITHMS                                                | ACTION REQUIRED                      | TIMELINE                                                                                       |
|------------|-------------|-----------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------|
| CRITICAL   | 90 - 100    | RSA-2048, RSA-1024, DES, 3DES, MD5, SHA-1, None/Plaintext | Immediate migration to ML-KEM-768    | Now                                                                                            |
| HIGH       | 60 - 89     | ECDSA P-256, ECDH, DSA, RSA-3072                          | Schedule migration within 6 months   | 6 months                                                                                       |
| MEDIUM     | 30 - 59     | RSA-4096, ECDSA P-384, X25519 (hybrid candidate)          | Plan migration, monitor NIST updates | 12 months                                                                                      |
| SAFE       | 0 - 29      | ML-KEM-768, ML-DSA-65, SLH-DSA, AES-256, SHA-3            | No action needed                     | Compliant  |

## 13 Test Acceptance Criteria Checklist

Use this master checklist to sign off on your testing session:

### Functional Tests

- ☐ **F-001:** Dashboard loads and displays 3 pre-seeded services with correct risk levels
- ☐ **F-002:** Risk score ring chart renders with correct value and color
- ☐ **F-003:** SBOM ↔ CBOM Bridge displays correlated CVE-to-cipher data
- ☐ **F-004:** AI Advisor generates a well-structured remediation playbook
- ☐ **F-005:** Policy migration (live mode) upgrades 2 vulnerable services to ML-KEM-768

- ☐ **F-006:** Dashboard auto-refreshes post-migration without manual page reload
- ☐ **F-007:** Compliance report opens in new tab with correct framework evaluations
- ☐ **F-008:** All API endpoints return valid JSON responses

### Post-Migration Verification

- ☐ **M-001:** Risk score = 0 after migration
- ☐ **M-002:** All services show ML-KEM-768 / SAFE
- ☐ **M-003:** Compliance badges change from WARN to PASS
- ☐ **M-004:** Re-running migration correctly reports "No vulnerable workloads"

### Performance & UX

- ☐ **P-001:** API health check responds within 500ms
- ☐ **P-002:** CBOM data loads within 2 seconds on page open
- ☐ **P-003:** Terminal log animation plays smoothly (no stuttering)
- ☐ **P-004:** All interactive elements have hover/click feedback

### Data Ingestion (Optional)

- ☐ **D-001:** Custom CBOM entries appear in dashboard after ingestion
- ☐ **D-002:** Custom services can be migrated to post-quantum ciphers
- ☐ **D-003:** Database reset ( `docker compose down -v` ) clears all data

## 14 Resetting the Environment

---

## Soft Reset (Keep data, restart services)

```
./stop.sh  
./start.sh
```

## Hard Reset (Delete all data, start fresh)

```
docker compose down -v  
./start.sh
```

### WHEN TO HARD RESET

After testing the quantum migration, all services will already be upgraded. To re-test the migration flow, perform a hard reset to re-seed the original vulnerable data.

## 15 FAQ & Troubleshooting

---

Q

### Do I need Java or Maven installed?

**No.** Everything runs inside Docker containers. You only need Docker Desktop.

Q

## Can I test with my real Kubernetes cluster data?

**Yes.** Use Section 10 (Scenario F) to ingest your own service data via the API. In the future, the PQ-Sentry CLI scanner will automate this by scanning live cluster TLS configurations.

Q

## The dashboard shows "Failed to fetch" errors

The backend API isn't running or hasn't finished starting. Run `curl http://localhost:8080/api/actuator/health` to check. If it fails, run `docker logs pq-sentry-api` to see errors.

Q

## Port 8080 is already in use

Stop the other application using port 8080, or check what's using it: `lsof -i :8080` (macOS/Linux).

Q

## Can I run the migration more than once?

**Yes.** If all services are already migrated, it will report "No quantum-vulnerable workloads detected." To re-test, perform a hard reset: `docker compose down -v && ./start.sh`

Q

## Is this testing against a real quantum computer?

**No.** PQ-Sentry identifies cryptographic algorithms that would be vulnerable to quantum attacks and helps you migrate proactively, before quantum computers are available.

---

PQ-Sentry — Post-Quantum Cryptography Readiness Platform

v1.0.0 • Confidential • For Authorized Testers Only

© 2026 PQ-Sentry. All rights reserved. • [support@pqsentry.io](mailto:support@pqsentry.io)